



Formato de la memoria



La plantilla de la Memoria actualizada está publicada en:

<https://github.com/TTFA-TTFB/Plantilla-para-memoria>

Ahí se explica cómo se usa el LaTeX, cuál es el software recomendado (TexMaker, Overleaf o Crixet^[1]) y cuáles son los lineamientos para escribir la Memoria (manejo de figuras, referencias, etc.)

Formato

- **"negritas y texto subrayado":**
No usarlas en ningún párrafo

Formato

- **"uso de itálicas":**
Usarlas para definir siglas originadas en otro idioma. Por ejemplo, RTOS (del inglés, *Real Time Operating System*)

Formato

- **"la escritura debe ser impersonal":**
Por ejemplo, **no** usar "el diseño lo hice de acuerdo..."

Formato

- **"usar referencias bibliográficas del tipo [1] en los párrafos":**

Por ejemplo, “el diseño está basado en la EDU-CIAA [1]...”

Formato

- **"para referenciar imágenes usar notas al pie":**
Por ejemplo:

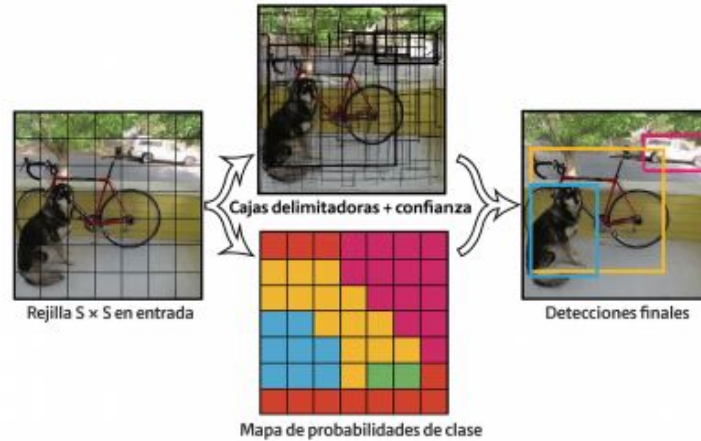


FIGURA 1.1. Esquema de la arquitectura YOLO.¹

las capas de atención multi cabeza, redes feed-forward y normalización que componen la base de modelos multimodales modernos.

¹Fuente: adaptado de [2].

Formato

- **"formato de tablas (único)":**

TABLA 3.1. Configuración de modelos LLM empleados, versión de API, tipo de tarea y costo aproximado.

Proveedor	Modelo	Uso principal	Costo estimado
OpenAI	GPT-4 (8K)	Redacción avanzada, comprensión profunda	\$0,03/\$0,06 por 1K tokens (in/out)
OpenAI	GPT-3.5 Turbo	Consultas frecuentes, prototipado	\$0,0015/\$0,002 por 1K tokens
Anthropic	Claude 3 Haiku	Redacción sintética, consultas interactivas	\$0,25 por 1M tokens aprox.
Google	Gemini Pro 1.5	Buen rendimiento general, bajo costo	Gratis en planes actuales (sujeto a cambios)

Formato

- **"ecuaciones":**

color YIQ utilizando la ecuación 3.1. Se **trabajo** en este espacio de color debido a su capacidad para separar la información de luminancia (Y) de la crominancia (I y Q), lo que permite una mejor diferenciación de tonalidades sin verse afectado por variaciones de iluminación.

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0,299 & 0,587 & 0,114 \\ 0,596 & -0,274 & -0,322 \\ 0,211 & -0,523 & 0,312 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (3.1)$$

Formato

- **"código":**
 - **No** agregar código.
 - Si **excepcionalmente** se necesita, usar el formato de la plantilla.
 - Se pueden sumar diagramas de flujo, estado y/o secuencia en lugar de código.

Formato

- **"letra monoespaciada":**
 - Usarla para indicar nombres propios de la implementación, como nombres de funciones, bibliotecas, archivos, directorios o variables.
 - Se pueden usar `\texttt{}`, `\verb|` o `\begin{verbatim}`.

Finalmente, tras completar la depuración y corrección de las geometrías vectoriales, se procedió a su conversión al formato raster utilizando la herramienta Rasterize (Vector to Raster) de QGIS. Se mantuvo la resolución de 10 metros por píxel para conservar la alineación espacial con las imágenes originales. Los píxeles correspondientes a las áreas de cultivo fueron codificados con



Errores comunes en el formato de la memoria



Errores comunes

- **"problemas de unificación de criterio":**
 - cifras en letras y en números,
 - epígrafes y enumeraciones con y sin punto final,
 - pronombres y adverbio "solo" con y sin tildes.

Errores comunes

Antes

- Desarrollo de software del algoritmo PID
- Desarrollo de Software de la interfaz gráfica
- Desarrollo de Hardware del sistema embebido
- Desarrollo del demo demostrativo del proceso nivel
- Integración del software del microcontrolador con la interfaz gráfica
- Integración de los sensores de medición con el hardware embebidos
- Integración del hardware embebido con el demo nivel

Después

- Desarrollo de software del algoritmo PID
- Desarrollo de software de la interfaz gráfica
- Desarrollo de hardware del sistema embebido
- Desarrollo del proceso real de nivel
- Integración del software del microcontrolador con la interfaz gráfica
- Integración de los sensores de medición con el hardware embebido
- Integración del hardware embebido con el proceso de nivel del tanque

Errores comunes

- **"mezclar uso de *proyecto* y *trabajo*":**
 - en la planificación se habla de ***proyecto*** y "en futuro" porque aún no está hecho,
 - en la memoria es un ***trabajo*** concluido y se usa el pasado: "se implementó un...".
 - en la memoria se usa "En este capítulo se presenta...", "En la figura X se puede observar", etc.

Errores comunes

Antes

Como se detalla en la sección 1.3, si bien se disponía del hardware y del servidor web, aún no se contaba con el firmware necesario para controlar el equipo y establecer la comunicación con el servidor. El objetivo del proyecto aquí detallado fue el desarrollo de dicho firmware.

Después

Como se detalla en la sección 1.3, si bien se disponía del hardware y del servidor web, aún no se contaba con el firmware necesario para controlar el equipo y establecer la comunicación con el servidor. El objetivo del trabajo aquí detallado fue el desarrollo de dicho firmware.

Errores comunes

- **"uso incorrecto de imágenes":**
 - utilizar texto ilegible,
 - no respetar los márgenes,
 - no utilizar notas al pie para referencia.

Errores comunes

Antes

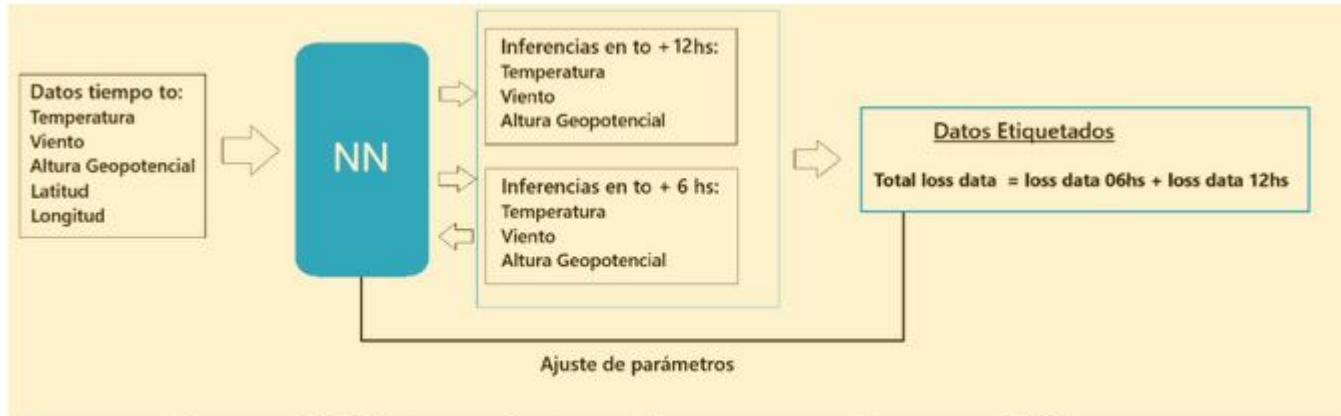


FIGURA 3.9. Entrenamiento en dos pasos en red neuronal *fully connected*.

Errores comunes

Después

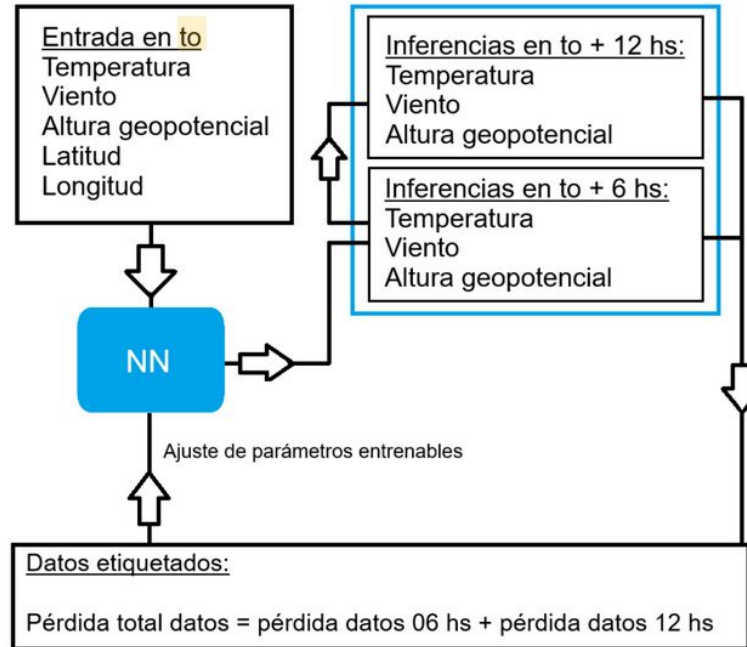
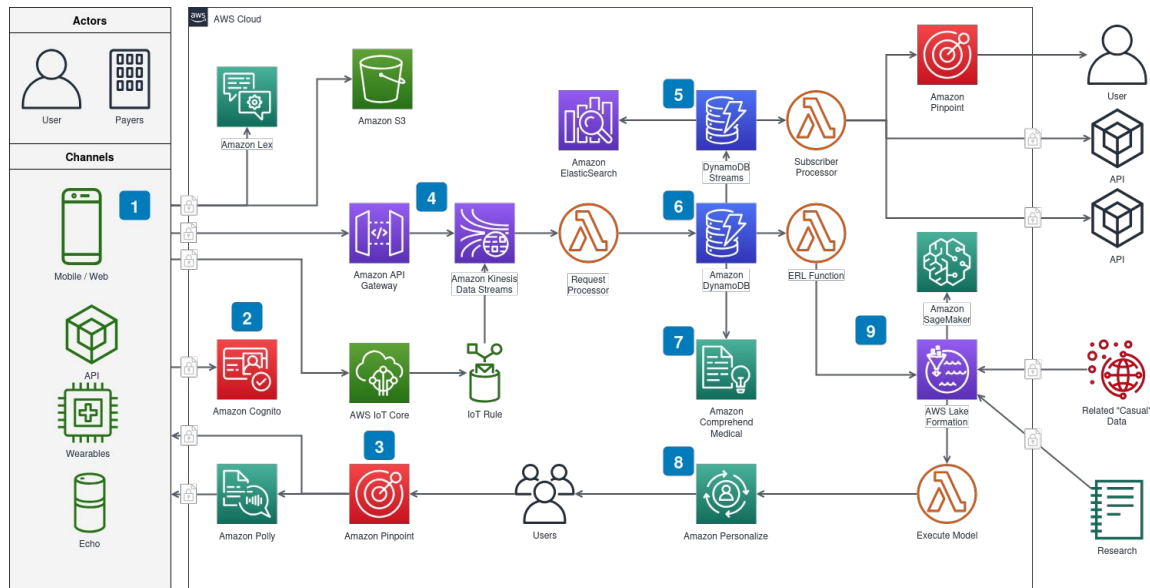


FIGURA 3.9. Entrenamiento en dos pasos en red neuronal *fully connected*.

(Enlace)



Errores comunes

Antes



FIGURA 4.29. Gateway LoRaWAN montado sobre la Raspberry Pi

Errores comunes

Después

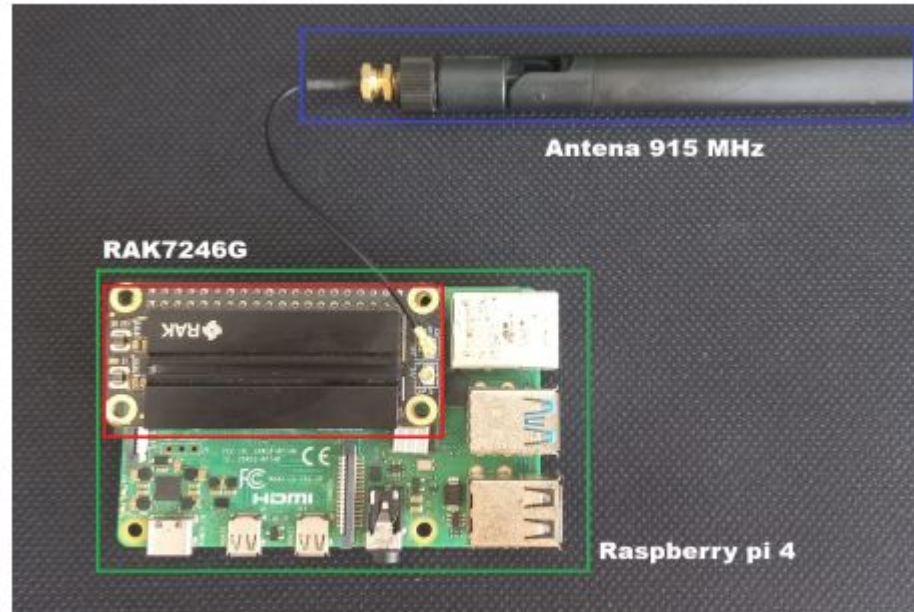


FIGURA 4.29. Gateway LoRaWAN montado sobre la Raspberry Pi 4.

Errores comunes

de referencia (impedancia). Comenzando del círculo azul del lado izquierdo, si se mueve el cursor hacia arriba o abajo se incrementa o decrementa el valor de la inductancia de la bobina que se debe colocar en serie en el circuito respectivamente. Partiendo del círculo rojo de en la parte derecha si se mueve el cursor hacia arriba o abajo se incrementa o decrementa el valor del capacitor en paralelo que se debe colocar en el circuito Pi. Al agregar un circuito Pi, que se compone de tres

Antes

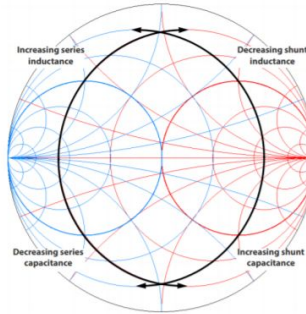


FIGURA 3.8: Manejo carta de Smith[16].

componentes paralelo, serie y paralelo como se puede observar en la figura 3.9, es posible cambiar la impedancia hasta 50Ω (al menos con dos componentes) y se tiene la opción de que se puede realizar la sintonización en cualquier momento y para cualquier caso de uso. es decir, si se tiene un dispositivo sintonizado para una aplicación específica pero el mismo dispositivo se quiere usar para colocarlo por ejemplo, sobre una superficie metálica, simplemente se debería realizar la sintonización de nuevo y no afectaría en una nueva fabricación de PCB.

Errores comunes

Después

El manejo de la carta de Smith es fundamental, ya que basados en un punto de referencia se obtienen los valores aproximados de las bobinas y/o capacitores que se deben colocar en el circuito Pi que garanticen que la antena queda en la frecuencia adecuada y con la mejor eficiencia. En la figura ver figura 3.9 se puede observar como es el desplazamiento en la carta de Smith partiendo de un punto de referencia (impedancia). Comenzando del círculo azul del lado izquierdo, si se mueve el cursor hacia arriba o abajo se incrementa o decremента el valor de la inductancia de la bobina que se debe colocar en serie en el circuito respectivamente. Partiendo del círculo rojo de en la parte derecha si se mueve el cursor hacia arriba o abajo se incrementa o decremента el valor del capacitor en paralelo que se debe colocar en el circuito Pi.

Al agregar un circuito Pi, que se compone de tres componentes paralelo, serie y paralelo como se puede observar en la figura 3.10, es posible cambiar la impedancia hasta 50Ω (al menos con dos componentes) y se tiene la opción de que se puede realizar la sintonización en cualquier momento y para cualquier caso de uso. Es decir, si se tiene un dispositivo sintonizado para una aplicación específica pero el mismo dispositivo se quiere usar para colocarlo por ejemplo, sobre una superficie metálica, simplemente se debería realizar la sintonización de nuevo y no afectaría en una nueva fabricación de PCB.

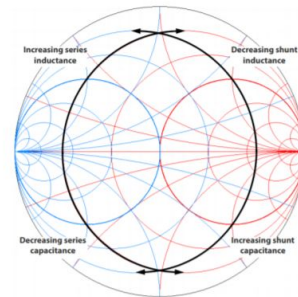


FIGURA 3.9: Manejo carta de Smith[23].

Errores comunes

- **"uso correcto del resumen introductorio":**
 - dos a tres líneas,
 - no lleva título propio.

Errores comunes

Antes

5

Capítulo 2

Introducción Específica

Falta resumen

2.1. Requerimientos

A continuación se enumeran los requerimientos del proyecto.

1. Requerimientos de documentación:

- 1.1 Se debe generar un Memoria Técnica con la documentación de ingeniería detallada.
- 1.2 Se debe generar un documento de casos de prueba.

2. Requerimientos funcionales del sistema:

- 2.1 El sistema debe adquirir datos de un array de sensores de temperatura a intervalos regulares con un periodo de adquisición seleccionable.
- 2.2 El sistema debe adquirir datos de un anemómetro a intervalos regulares con un periodo de adquisición seleccionable.
- 2.3 El sistema debe almacenar los datos de temperatura y velocidad de viento adquiridas junto con una marca de tiempo identificatoria en un medio físico no volátil.
- 2.4 El sistema debe poder operar con dos perfiles de consumo de energía, uno maximizando el desempeño y otro minimizando el consumo de energía.
- 2.5 El sistema debe contar con una interfaz serie tipo HMI cableada que permita interactuar y realizar operaciones de configuración y mantenimiento.

3. Requerimientos de verificación:

- 3.1 Se debe generar una matriz de trazabilidad entre la Memoria Técnica y los requerimientos.
- 3.2 Se debe generar una matriz de trazabilidad entre las pruebas de integración y los requerimientos.

4. Requerimientos de validación:

- 4.1 Se debe generar una matriz de trazabilidad entre el documento de casos de prueba y los requerimientos.

Después

5

Capítulo 2

Introducción Específica

En este capítulo se identifican aspectos relevante de la planificación. Asimismo, se describen las herramientas empleadas para la realización de este trabajo.

2.1. Requerimientos

A continuación se enumeran los requerimientos del proyecto:

1. Requerimientos de documentación:

- 1.1 Se debe generar un Memoria Técnica con la documentación de ingeniería detallada.
- 1.2 Se debe generar un documento de casos de prueba.

2. Requerimientos funcionales del sistema:

- 2.1 El sistema debe adquirir datos de un array de sensores de temperatura a intervalos regulares con un periodo de adquisición seleccionable.
- 2.2 El sistema debe adquirir datos de un anemómetro a intervalos regulares con un periodo de adquisición seleccionable.
- 2.3 El sistema debe almacenar los datos de temperatura y velocidad de viento adquiridas junto con una marca de tiempo identificatoria en un medio físico no volátil.
- 2.4 El sistema debe poder operar con dos perfiles de consumo de energía: máximo desempeño y mínimo consumo de energía, respectivamente.
- 2.5 El sistema debe contar con una interfaz serie cableada que permita realizar operaciones de configuración y mantenimiento.

3. Requerimientos de verificación:

- 3.1 Se debe generar una matriz de trazabilidad entre la Memoria Técnica y los requerimientos.
- 3.2 Se debe generar una matriz de trazabilidad entre las pruebas de integración y los requerimientos.

4. Requerimientos de validación:

- 4.1 Se debe generar una matriz de trazabilidad entre el documento de casos de prueba y los requerimientos.

Errores comunes

- **"no introducir/desarrollar el contenido":**
 - listados, tablas,
 - imágenes.

Errores comunes (no introducir listados, tablas o imágenes)

Antes

Entender cómo las condiciones experimentales afectan a la partícula de sílice es referirse a cómo varía el tamaño de esta, lo que permitirá el uso adecuado para la industria que la requiera. Las condiciones experimentales para el análisis del tamaño de la partícula, serán el **TEOS**, concentración de agua y **amoníaco** como se describe en varias fuentes científicas utilizadas como almacén de datos para el desarrollo de esta investigación.



FIGURA 1.1. Gel de Sílice

Errores comunes (no introducir listados, tablas o imágenes)

Después

Una presentación de gel muy común, son los sobres absorbentes de humedad presentes a la hora de comprar un par de zapatos en caja como se muestra en la figura 1.1.



FIGURA 1.1. Gel de sílice.

Errores comunes (no introducir listados, tablas o imágenes)

Antes

1.4.3. Alcance

- Análisis y modelado de datos experimentales.
- Modelado de parámetros para de técnicas estadísticas y de machine learning.

Errores comunes (no introducir listados, tablas o imágenes)

Después

1.4.3. Alcance

El alcance de este trabajo incluye las siguientes actividades y consideraciones:

- Análisis y modelado de datos experimentales.

Errores comunes (no introducir listados, tablas o imágenes)

Lo correcto

Para finalizar, la elección de los aceleradores debe considerar los parámetros clave para aplicaciones de borde. La tabla 1.2 presenta el análisis comparativo de dos métricas fundamentales: capacidad computacional, medida en tera operaciones por segundo (TOPS) y consumo energético, medido en vatios (W). En esta tabla se observa una diferencia clave entre arquitecturas. Mientras las GPU ofrecen una capacidad de cómputo máxima, los ASIC especializados presentan un consumo significativamente menor. Esto se traduce en una mejor relación TOPS/W, un parámetro relevante en dispositivos con alimentación limitada.

TABLA 1.2. Comparativa de modelos de aceleradores de *deep learning*.

Fabricante	Modelo	Capacidad (TOPS)	Consumo (W)
Nvidia	Orin Nano 8GB [2]	40	15
Nvidia	Orin NX [3]	100	25
Nvidia	AGX Orin [4]	275	60
Hailo	8 [5]	26	2,5
Hailo	8L [6]	13	1,5
Google	Coral Edge [7]	4	2
Mythic	M1076 [8]	25	4
Quadric	Chimera Nano [9]	7	No especificado
ARM	ETHOS-U85 [10]	4	No especificado
Altera	Agilex 5E [11]	26	No especificado

Errores comunes

"Formato incorrecto de tablas":

Errores comunes (no introducir listados, tablas o imágenes)

Después

Antes

Etapa	Horas	Hitos
Documentación y análisis preliminar	100	Plan de trabajo
		Presentación de plan de trabajo
Diseño e implementación	340	Documentación de submódulos
Verificación y validación	60	Reporte de pruebas unitarias
		Reporte de pruebas de integración
		Reporte de resultados de casos de prueba
Proceso de cierre	100	Memoria Técnica
		Presentación de Trabajo Final

TABLA 2.1: Etapas principales del proyecto con el detalle de las horas planificadas y los hitos a alcanzar en cada una de ellas.

TABLA 2.1: Etapas principales del proyecto. Se detallan de las horas planificadas y los hitos a alcanzar en cada etapa.

Etapa	Horas	Hitos
Documentación y análisis	100	Plan de trabajo
		Presentación de plan de trabajo
Diseño e implementación	340	Documentación de submódulos
Verificación y validación	60	Reporte de pruebas unitarias
		Reporte de pruebas de integración
		Reporte de casos de prueba
Proceso de cierre	100	Memoria Técnica
		Presentación de Trabajo Final

Errores comunes

"Formato incorrecto de referencias":

Errores comunes

Antes

Capítulo 1

Introducción General

En este capítulo se presenta el contexto y justificación de este trabajo final (10), la motivación para llevarlo a cabo (10), sus objetivos y alcance (10).

1.1. Contexto y justificación

Programarlas
todas son
distintas

En la actualidad existe una enorme variedad de plataformas de sistemas embebidos en el mercado, y si bien todas cuentan con dispositivos programables con características similares y periféricos compatibles, se observa que a la hora de programarlas todas son distintas. En consecuencia, el profesional desarrollador de sistemas embebidos debe invertir mucho tiempo en aprender a programar cada nueva plataforma.

Estas diferencias se deben a que los fabricantes de los dispositivos y empresas asociadas poseen sus respectivas arquitecturas de hardware (tanto en núcleos de procesamiento, como periféricos). Cada periférico exhibe externamente la funcionalidad esperada, pero su configuración y uso es diferente. Si bien esto da a los fabricantes la posibilidad de diferenciarse frente a la competencia y a los desarrolladores elegir el dispositivo programable que más se adecue a un proyecto, también causa que el programador deba conocer en detalle la arquitectura en particular del dispositivo a la hora de utilizarlo.

Además, estas empresas en su mayoría se limitan a ofrecer información de bajo nivel para programar el hardware directamente (escribiendo registros), o bien, mediante sus propias bibliotecas escritas generalmente en lenguaje C, que están diseñadas con una gran dependencia de la arquitectura de hardware subyacente, es decir, carecen de abstracción del hardware.

Por otro lado, se están portando cada vez más lenguajes de programación a las plataformas de sistemas embebidos que antes se reservaban para las computadoras de propósito general. Esto se debe a que las nuevas plataformas poseen cada vez más poder de procesamiento y memoria, a causa de la complejidad creciente de las aplicaciones deseadas. En consecuencia se utilizan lenguajes de programación más modernos, en parte por su mayor capacidad de abstracción, y en parte para estandarizar el lenguaje de programación de una aplicación en la que intervienen sistemas embebidos y computadoras de propósito general, siendo un ejemplo típico una aplicación de IoT¹.

¹Del inglés *Internet of Things*, es decir, Internet de las cosas, donde intervienen en una aplicación sistemas embebidos distribuidos, infraestructura de red y servidores.

Después

Capítulo 1

Introducción General

En este capítulo se presenta el contexto y justificación de este trabajo final, la motivación para llevarlo a cabo, sus objetivos y alcance.

1.1. Contexto y justificación

En la actualidad existe una enorme variedad de plataformas de sistemas embebidos en el mercado, y si bien todas cuentan con dispositivos programables con características similares y periféricos compatibles, se observa que a la hora de programarlas todas son distintas. En consecuencia, el profesional desarrollador de sistemas embebidos debe invertir mucho tiempo en aprender a programar cada nueva plataforma.

Estas diferencias se deben a que los fabricantes de los dispositivos y empresas asociadas poseen sus respectivas arquitecturas de hardware (tanto en núcleos de procesamiento, como en periféricos). Cada periférico exhibe externamente la funcionalidad esperada, pero su configuración y uso es diferente. Si bien esto da a los fabricantes la posibilidad de diferenciarse frente a la competencia y a los desarrolladores elegir el dispositivo programable que más se adecue a un proyecto, también causa que el programador deba conocer en detalle la arquitectura en particular del dispositivo a la hora de utilizarlo.

Además, estas empresas en su mayoría se limitan a ofrecer información de bajo nivel para programar el hardware directamente (escribiendo registros), o bien, mediante sus propias bibliotecas escritas generalmente en lenguaje C, que están diseñadas con una gran dependencia de la arquitectura de hardware subyacente, es decir, carecen de abstracción del hardware.

Por otro lado, se están portando cada vez más lenguajes de programación a las plataformas de sistemas embebidos, que antes se reservaban para las computadoras de propósito general. Esto se debe a que las nuevas plataformas poseen cada vez más poder de procesamiento y memoria, a causa de la complejidad creciente de las aplicaciones deseadas. En consecuencia se utilizan lenguajes de programación más modernos, en parte por su mayor capacidad de abstracción, y en parte para estandarizar el lenguaje de programación de una aplicación en la que intervienen sistemas embebidos y computadoras de propósito general, siendo un ejemplo típico una aplicación de IoT¹.

¹Del inglés *Internet of Things*, es decir, Internet de las cosas, donde intervienen en una aplicación sistemas embebidos distribuidos, infraestructura de red y servidores.

Errores comunes

Antes

1.1. Agricultura de Precisión

La agricultura de precisión es una estrategia de gestión que recoge, procesa y analiza datos temporales, espaciales e individuales y los combina con otras informaciones para respaldar las decisiones de manejo de acuerdo con la variabilidad estimada, y así mejorar la eficiencia en el uso de recursos de la producción agrícola.

La agricultura de precisión no es más que la consecuencia del desembarco de las tecnologías de la información y la comunicación en la agricultura o agroTICs. Las agroTICs permiten la adquisición de datos del cultivo y de su medio, el procesamiento de esos datos para convertirlos en información útil y el tratamiento y almacenamiento de esa información para la toma de decisiones y actuaciones. Disponer de información detallada sobre características del cultivo y de su medio (el suelo, el clima, entre otros factores) ayudan al agricultor y al técnico a tomar decisiones mejor fundamentadas. Todo agricultor sabe que sus campos no producen exactamente lo mismo en toda su superficie. Aun así, no es hasta que se cuantifican esas diferencias en la productividad que se ve la magnitud que puede llegar a tener dicha variabilidad. El tener en cuenta esa variabilidad en el manejo de las parcelas es el objetivo de la agricultura de precisión. Esa información condujo a la intuición que algunos campos deben manejarse no de forma uniforme sino de forma específica atendiendo a las características de las diferentes zonas que lo componen. Este tipo de manejo se denomina manejo específico localizado (del inglés *Site-Specific Management*) y consiste en realizar labores agrícolas o aplicaciones de insumos de forma variable en distintas partes de la parcela que así lo requieran. Más aún, en determinados casos, tales como en los cultivos arbóreos, se puede llegar al manejo de las explotaciones a nivel de planta individual.

En resumen, la agricultura de precisión consiste en realizar la operación adecuada, en el lugar adecuado, en el momento adecuado, de la manera adecuada y en la cantidad adecuada. Y para ello se necesitan los siguientes elementos:

- Observaciones visuales o mediante sensores que permitan la toma de datos georeferenciados, es decir, con coordenadas que permitan su localización en la parcela.



REFERENCIAS

Errores comunes

Después

1.1. Agricultura de precisión

La Agricultura de Precisión es una estrategia de gestión que recoge, procesa y analiza datos temporales, espaciales e individuales y los combina con otras informaciones para respaldar las decisiones de manejo de acuerdo con la variabilidad estimada, y así mejorar la eficiencia en el uso de recursos, la productividad, la calidad, la rentabilidad y la sostenibilidad de la producción agrícola [1]. La agricultura de precisión es la consecuencia del desembarco de las tecnologías de la información y la comunicación en la agricultura o agroTICs. Las agroTICs permiten la adquisición de datos del cultivo y de su medio, el procesamiento de esos datos para convertirlos en información útil y el tratamiento y almacenaje de esa información para la toma de decisiones y actuaciones. Disponer de información detallada sobre características del cultivo y de su medio (el suelo, el clima, entre otros factores) ayudan al agricultor y al técnico a tomar decisiones mejor fundamentadas.

Todo agricultor sabe que sus campos no producen exactamente lo mismo en toda su superficie. Aun así, no es hasta que se cuantifican esas diferencias en la productividad que se ve la magnitud que puede llegar a tener dicha variabilidad. El tener en cuenta esa variabilidad en el manejo de las parcelas es el objetivo de la agricultura de precisión. Esa información condujo a la intuición que algunos campos deben manejarse no de forma uniforme sino de forma específica atendiendo a las características de las diferentes zonas que lo componen. Este tipo de manejo se denomina manejo específico localizado (del inglés *Site Specific Management*) y consiste en realizar labores agrícolas o aplicaciones de insumos de forma variable en distintas partes de la parcela que así lo requieran [2]. Más aún, en determinados casos, tales como en los cultivos arbóreos, se puede llegar al manejo de las explotaciones a nivel de planta individual.

En resumen, la agricultura de precisión consiste en realizar la operación adecuada, en el lugar adecuado, en el momento adecuado, de la manera adecuada y en la cantidad adecuada. Para esto se necesitaría inevitablemente la tecnología que se describe a continuación, mas alla de que la experiencia del agricultor aporta como en cualquier **area** donde se junta la experiencia de una persona con la tecnología [3].

Errores comunes

- **"uso correcto de los números de línea en las porciones de código"**
- **No presentar código**

Errores comunes

3.7. Servidor web

Para el desarrollo del prototipo se elaboró una página web que se almacena en la memoria flash del ESP32 y que se puede acceder a través de una conexión de área local gracias a la conexión Wi-Fi de la que dispone el módulo. En el código 3.3 se muestra el pseudocódigo donde se encuentran las principales funciones que se utilizan para la conexión Wi-Fi y para la creación del servidor web.

```
1 main () {
2     //...
3     iniciarWiFi();
4     iniciarServidorWeb();
5     //...
6 }
7
8 //...
9
10 void iniciarWiFi()
11 {
12     esp_wifi_init(); //Inicializa el stack de Wi-Fi con configuraciones
13     //por defecto
14     esp_wifi_set_mode(ESTACION); //Configura el modo estacion
15     esp_wifi_set_config(wifi_config); //Configura nombre de red del AP y
16     //contraseña
17     esp_wifi_start(); //Realiza la conexión Wi-Fi
18     registraHandlerDeEventos();
19 }
20
21 iniciarServidorWeb()
22 {
23     if (httpd_start(configuraciones) == OK) //Crea un servidor HTTP
24     {
25         httpd_register_uri_handler(paginaHandler); //registra una pagina
26     }
27 }
28
29 paginaHandler() //evento que se ejecuta cuando alguien accede a la
30 //pagina
31 {
32     //Codigo HTML de la pagina que contiene el volumen en el punto final
33     //y una tabla del volumen inyectado y pH
34 }
```

CÓDIGO 3.3. Pseudocódigo del servidor web.

Después

3.7. Servidor web

Para el desarrollo del prototipo se elaboró una página web que se almacena en la memoria flash del ESP32 y que se puede acceder a través de una conexión de área local gracias a la conexión Wi-Fi de la que dispone el módulo. En el código 3.4 se muestra el pseudocódigo donde se encuentran las principales funciones que se utilizan para la conexión Wi-Fi y para la creación del servidor web.

La función `iniciarWiFi` de la línea 10 se encarga de habilitar la conexión Wi-Fi en modo estación, con el nombre de la red y la contraseña del punto de acceso al cual se desea conectar. En la línea 16 registra eventos que se ejecutan cuando hay un cambio de ip o se produce un corte en la conexión, y tienen como objetivo restablecer la conexión.

La función `iniciarServidorWeb` de la línea 20 inicia un servidor HTTP con las configuraciones por defecto que ofrece la API y registra un evento asociado a la página web. Este evento se ejecuta cuando un cliente, conectado a la red local, intenta establecer la conexión mediante la ip asociada al ESP32. El controlador del evento ejecuta la función de la línea 28 que actualiza la página web con los datos obtenidos en el último proceso de titulación.

28

Capítulo 3. Diseño e implementación

```
1 main () {
2     //...
3     iniciarWiFi();
4     iniciarServidorWeb();
5     //...
6 }
7
8 //...
9
10 void iniciarWiFi()
11 {
12     esp_wifi_init(); //Inicializa el stack de Wi-Fi con configuraciones
13     //por defecto
14     esp_wifi_set_mode(ESTACION); //Configura el modo estacion
15     esp_wifi_set_config(wifi_config); //Configura nombre de red del AP y
16     //contraseña
17     esp_wifi_start(); //Realiza la conexión Wi-Fi
18     registraHandlerDeEventos();
19 }
20
21 iniciarServidorWeb()
22 {
23     if (httpd_start(configuraciones) == OK) //Crea un servidor HTTP
24     {
25         httpd_register_uri_handler(paginaHandler); //registra una pagina
26     }
27 }
28
29 paginaHandler() //evento que se ejecuta cuando alguien accede a la
30 //pagina
31 {
32     //Codigo HTML de la pagina que contiene el volumen en el punto final
33     //y una tabla del volumen inyectado y pH
34 }
```

CÓDIGO 3.4. Pseudocódigo del servidor web.

Herramienta recomendada



PlantUML

([Enlace](#))



Su código



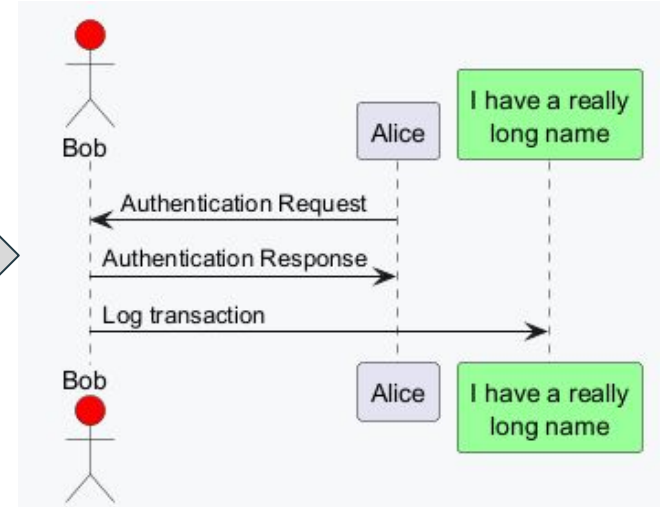
Su IA
favorita



```
@startuml
Alice -> Bob: Authentication Request
Bob --> Alice: Authentication Response

Alice -> Bob: Another authentication Request
Alice <-- Bob: Another authentication Response
@enduml
```

Código UML



Errores comunes

- **"uso correcto de imágenes con texto en otro idioma"**
 - Hay que traducir lo posible

Antes

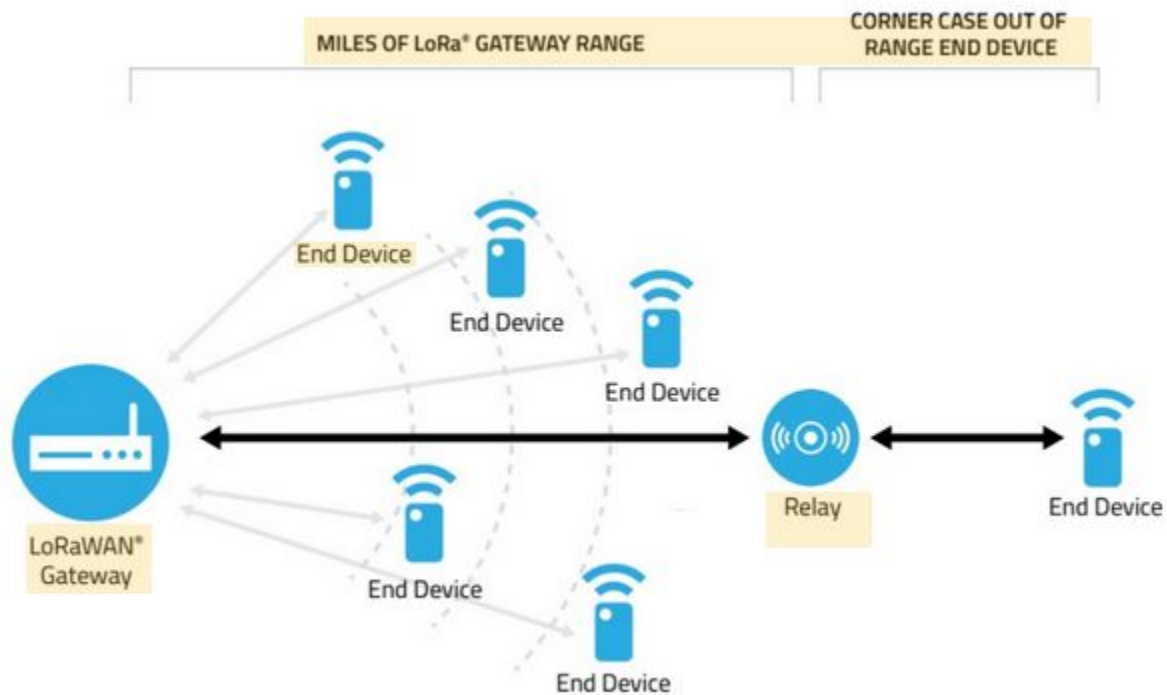


FIGURA 3.2. Funcionalidad del *relay* en LoRaWAN ¹.

Después

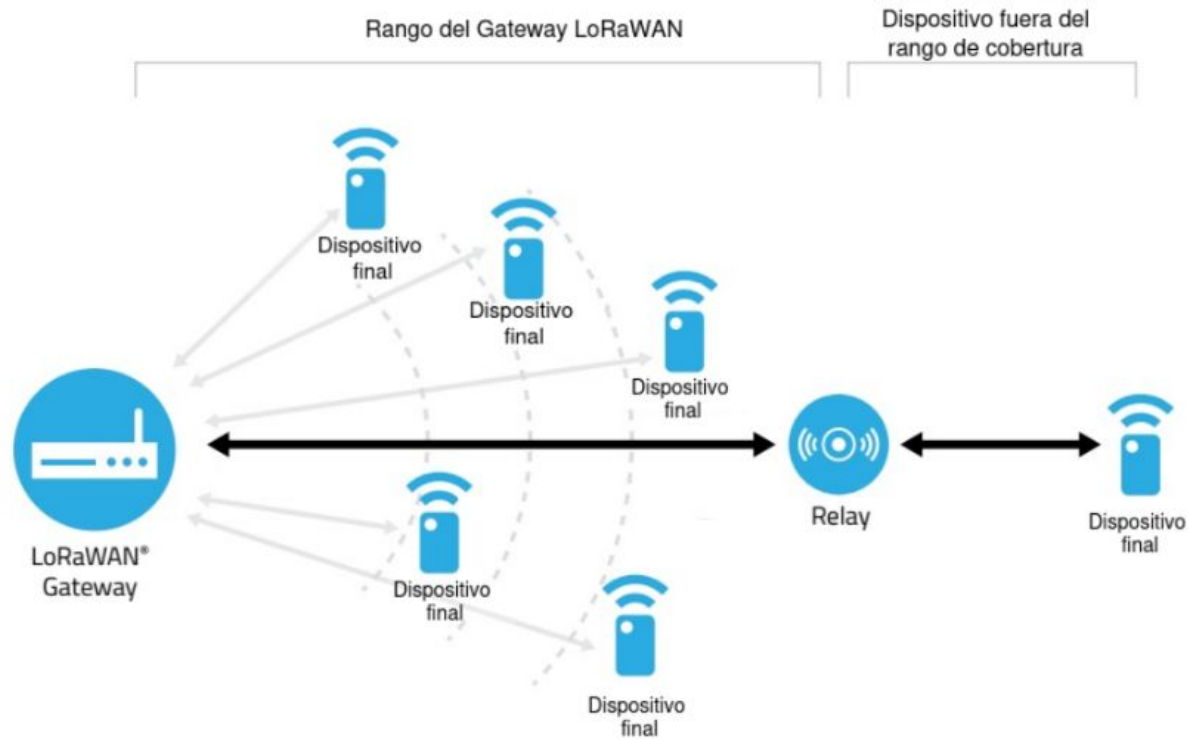


FIGURA 3.2. Funcionalidad del *relay* en LoRaWAN ¹.

Errores comunes

- **"errores de Chat GPT"**
 - En resumen, en conclusión, en conjunto, en síntesis, en contraste...
 - abuso del gerundio,
 - referencias falsas,
 - uso de guiones (separador 2º jerarquía),
 - redacción personal.

Errores comunes

- **Función de control de procesos:** se desarrolló una función independiente para monitorear las etapas críticas del pipeline —descarga de audios, transcripción y análisis de sentimiento—. Esta función permite verificar que cada proceso se ejecute de manera correcta y completa e identifique fallos o interrupciones sin formar parte del flujo principal de procesamiento. De este

Errores comunes

razones. En primer lugar, se hizo un uso eficiente de modelos preentrenados como Whisper y RoBERTa, disponibles libremente y validados por la comunidad científica, lo que permitió evitar los elevados costos y tiempos de entrenamiento desde cero, **acelerando** así la implementación. Además, estos modelos permiten aprovechar la transferencia de aprendizaje, ya que fueron entrenados sobre grandes corpus multilingües y de redes sociales, **logrando** un buen desempeño sin necesidad de un reentrenamiento. Por último, se priorizó la flexibilidad y adaptabilidad del sistema, dejando abierta la posibilidad de incorporar otros modelos

Errores comunes

En conclusión, si bien el estado del arte muestra un abanico de soluciones maduras orientadas a la gestión remota y a la comunicación bidireccional, la mayoría de ellas se ve condicionada por altos costos, rigidez tecnológica o requerimientos de infraestructura poco compatibles con entornos de conectividad limitada. Estas restricciones ponen en evidencia un vacío que habilita la exploración de alternativas adaptadas a las particularidades del ámbito vial en Argentina.

Errores comunes

En conclusión, si bien el estado del arte muestra un abanico de soluciones maduras orientadas a la gestión remota y a la comunicación bidireccional, la mayoría de ellas se ve condicionada por altos costos, rigidez tecnológica o requerimientos de infraestructura poco compatibles con entornos de conectividad limitada. Estas restricciones ponen en evidencia un vacío que habilita la exploración de alternativas adaptadas a las particularidades del ámbito vial en Argentina.